

LEAD University

Programación Paralela y Distribuida

Tarea 1

Juego de la Vida de Conway

Estudiante:
Susana Herrera

Profesor:
Johansell Villalobos Cubillo

Fecha: 2026-05-26

1. Descripción del Proyecto

El Juego de la Vida de Conway es un autómata celular propuesto por el matemático británico John Horton Conway en 1970. El algoritmo fue puesto en práctica en Python en esta tarea, empleando programación dirigida a objetos. Se observaron patrones tradicionales y se midió el rendimiento a través de diferentes métodos.

2. Implementación Orientada a Objetos

2.1 Clase GameOfLife

Se desarrolló la clase GameOfLife en el archivo `game_of_life.py` con los siguientes métodos:

- **`__init__(self, rows, cols, initial_state=None)`** — inicializa el tablero con estado aleatorio o provisto manualmente.
- **`step(self)`** — avanza una generación aplicando las 4 reglas de Conway simultáneamente.
- **`run(self, steps)`** — ejecuta múltiples iteraciones y retorna el tiempo total de ejecución.
- **`get_state(self)`** — devuelve una copia del estado actual del tablero.
- **`set_pattern(self, pattern, row, col)`** — inserta un patrón clásico en una posición específica.

2.2 Estrategia de cálculo con NumPy

El núcleo del algoritmo utiliza `np.roll` para desplazar la grilla en las 8 direcciones vecinas y sumaras, obteniendo el conteo de vecinos de todas las celdas en una sola operación vectorizada. Esto evita bucles Python celda por celda y delega el cómputo a rutinas optimizadas en C dentro de NumPy.

2.3 Tablero toroidal

Los bordes: el lado derecho se conecta con el izquierdo, y el borde de arriba se une con el de abajo. Esto se consigue automáticamente con `np.roll`, que permite la rotación de la indexación evitando requerir una gestión específica de los bordes.

2.4 Patrones clásicos incluidos

- **Glider** — nave espacial que se desplaza diagonalmente.
- **Blinker** — oscilador que alterna entre orientación horizontal y vertical.
- **Toad** — oscilador de periodo 2 con forma cambiante.
- **Block** — estructura estática que no cambia nunca.
- **Beacon** — oscilador que parpadea en las esquinas.

3. Visualización

La visualización se hizo en el archivo `visualizer.py` utilizando `matplotlib`. Se generaron tres tipos de salidas:

- **patterns_showcase.png** — panel con los 5 patrones clásicos, cada uno capturado en 4 generaciones distintas para observar su evolución.
- **glider_detail.png** — evolución del Glider en 6 momentos sobre una grilla de 30x30, mostrando claramente su desplazamiento diagonal.
- **random_evolution_64.png** — evolución de un estado inicial aleatorio en una grilla de 64x64, mostrando cómo el sistema se estabiliza con el tiempo.

El esquema de colores utiliza fondo negro con celdas vivas en verde neón (#39ff14), para darle un poco de nostalgia y recordar los monitores de la era en que fue creado el juego.

4. Resultados de Rendimiento

4.1 Tabla de tiempos medidos

Se realizaron 3 repeticiones por tamaño, tomando la mediana de 20 pasos medidos para reducir el ruido del sistema operativo:

Grilla	Celdas	ms/iter (mediana)	Memoria estimada
32×32	1,024	0.1507 ms	~0.002 MB
64×64	4,096	0.1546 ms	~0.008 MB
128×128	16,384	0.1745 ms	~0.031 MB
256×256	65,536	0.2489 ms	~0.125 MB
512×512	262,144	0.6767 ms	~0.500 MB
1024×1024	1,048,576	2.9874 ms	~2.000 MB

4.2 Análisis de la complejidad empírica

La gráfica log-log nos muestra una pendiente empírica de aproximadamente 0.41, lo que indica que el tiempo de ejecución crece sublinealmente con respecto al número de celdas. Este resultado puede explicarse por dos factores:

- Caché del CPU: para grillas pequeñas (hasta $\sim 256 \times 256$), los datos caben completamente en la memoria caché L2/L3 del procesador. Las operaciones de `np.roll` son extremadamente rápidas porque no hay accesos a RAM principal.
- Paralelismo SIMD interno de NumPy: las operaciones vectorizadas aprovechan instrucciones AVX/SSE del procesador moderno, procesando múltiples celdas por ciclo de reloj.

Cuando el tamaño supera la caché (512×512 en adelante), el tiempo comienza a crecer de manera más pronunciada por los fallos de caché (cache misses), aunque aún así se mantiene muy por debajo de $O(n)$.

4.3 Complejidad de memoria

La implementación utiliza arrays `uint8` (1 byte por celda). El uso de memoria es $O(n)$ donde n es el número de celdas:

- Grilla principal: `rows x cols` bytes.
- Grilla temporal de vecinos: `rows x cols` bytes adicionales durante `step()`.
- Total para 1024×1024 : aproximadamente 2 MB, muy eficiente.

5. Análisis y Discusión

5.1 Escalabilidad

La implementación escala muy bien en tiempo gracias al uso de NumPy. Una grilla 1024×1024 (más de un millón de celdas) tarda menos de 3 ms por iteración, lo que permite simulaciones en tiempo real a más de 300 generaciones por segundo.

En cuanto a memoria, el crecimiento es estrictamente lineal ($O(n)$). Una grilla de 1024×1024 usa apenas 2 MB, por lo que el límite práctico no es la memoria sino el tiempo de cómputo para grillas mayores a 4096×4096 .

5.2 Cuellos de botella observados

- Fallos de caché: el cuello de botella principal para grillas grandes. `np.roll` crea copias del array internamente, lo que genera presión sobre la memoria cuando los datos no caben en caché.
- Overhead de Python: aunque NumPy es muy rápido, hay un costo fijo por llamada a función Python (~ 0.15 ms visible en los tamaños pequeños donde el tiempo no crece con el tamaño).

- Copias de array: cada llamada a `step()` genera arrays temporales para los 8 desplazamientos y el resultado. Esto podría optimizarse con buffers preasignados.

5.3 Comparación con complejidad teórica

Teóricamente, el Juego de la Vida tiene complejidad $O(n)$ ya que cada celda se evalúa exactamente una vez por generación. La pendiente empírica, que es 0.41 (por debajo de 1), se origina por el paralelismo interno de NumPy: efectivamente, en el nivel del hardware (SIMD), procesa varias celdas simultáneamente, lo cual hace que la complejidad práctica sea mejor que la teórica secuencial.

6. Conclusiones

- La vectorización es el método más eficaz para acelerar el cómputo numérico en Python, sin bucles explícitos. Así lo demuestra la implementación con NumPy.
- El Juego de la Vida, creado por Conway, es un ejemplo emblemático de emergencia: a partir de cuatro reglas sencillas se producen conductas complejas como osciladores, estructuras estables y naves espaciales.
- El paralelismo SIMD del hardware contemporáneo y la utilización eficaz de la caché del CPU hicieron que la complejidad empírica resultara sublineal.
- La memoria aumenta de manera lineal conforme se incrementa la cantidad de celdas, lo que permite que su implementación sea factible para grillas grandes.
- Con el fin de preservar la eficiencia en tiempo real, para las rejillas que superen las dimensiones 2048x2048 se aconseja examinar CuPy (GPU) o Numba (JIT compilation).

7. Referencias Bibliográficas

Numberphile. (2014, abril 4). *John Conway: Inventing the Game of Life* [Video]. YouTube. <https://www.youtube.com/watch?v=E8kUJL04ELA>

Computerphile. (2014, agosto 22). *Conway's Game of Life* [Video]. YouTube. <https://www.youtube.com/watch?v=ouipbDkwHWA>

Clear Code. (2024, marzo 5). *Conway's Game of Life tutorial in Python & Pygame - OOP* [Video]. YouTube. <https://www.youtube.com/watch?v=uR0INADr4dc>

Neuralnine. (2022, abril 29). *Conway's Game of Life in Python* [Video]. YouTube. <https://www.youtube.com/watch?v=cRWg2SWuXtM>

Python Simulation. (2023, octubre 1). *Python Simulation Tutorial - Conway's Game of Life* [Video]. YouTube. <https://www.youtube.com/watch?v=YDKuknw9WGs>

Anthropic. (2026). *Claude Sonnet 4.6* [Herramienta de inteligencia artificial]. Anthropic. <https://claude.ai>